The Partial Knowledge Paradox

When the Agent Reads the Skill and Ignores It
2 Variants × 3 Runs on Spring PetClinic (Boot 4)

Code Coverage Experiment v3

Dr. Mark Pollack

April 2026

Contents

1 What We Ran 2

2 The Smoking Gun 3

3 Two-Axis Results 4

4 Four Key Findings 5

5 The Behavioral Signature 6

6 The Exemplar Effect: v2 vs. v3 8

7 What This Experiment Cannot Show 9

8 Implications for Practice 9

9 What Comes Next 10

10 The Short Summary 10

1 What We Ran

The question

The v2 experiment showed that skills reduce search overhead by 25 percentage points and prompt hardening is the single biggest quality driver. But v2 started from *zero* tests. What happens when the project already has tests — and those tests use older patterns?

The v3 experiment tests a specific hypothesis: **when existing code contradicts skill guidance, the agent follows the code.**

This is the *partial knowledge paradox*: the agent has access to correct knowledge (via skills) but produces suboptimal output because it mimics the patterns it finds in the existing codebase.

The task

Write JUnit 5 tests for Spring PetClinic (Boot 4.0.1) to improve code coverage from 65% to at least 85% line coverage. The project starts with 6 seed test files that compile and pass but use **older patterns**:

- `mockMvc.perform()` — the established MockMvc API. Spring Framework 7 (which Boot 4 picks up) introduced `MockMvcTester`, a fluent replacement with better AssertJ integration. Both APIs work; `MockMvcTester` is the recommended path forward.
- No `flush()`/`clear()` in `@DataJpaTest` classes — a JPA testing best-practice gap. Without flushing and clearing the persistence context between writes and reads, assertions hit the Hibernate L1 cache rather than the database, masking potential bugs.

Skills explicitly teach the improved patterns. The seed files demonstrate the older ones.

The setup

- **Model**: Claude Sonnet 4.6
- **Task**: improve coverage on spring-petclinic-partial (Boot 4.0.1)
- **Baseline**: 64.9% line coverage (6 seed test files)
- **Coverage target**: $\geq$85% via JaCoCo
- **Build**: `./mvnw clean test jacoco:report`
- **Runs**: 6 sessions total (N=3 per variant)
- **Skills**: `spring-mvc-testing`, `spring-jpa-testing`, `spring-testing-fundamentals` (installed globally)
- **Tracking**: every tool call logged via `agent-journal`

The two variants

This is not a 7-variant ladder. It is a controlled pair designed to isolate one variable: prompt structure.

Variant	What the agent has
`simple`	Two-line prompt. "Add JUnit tests to improve code coverage." No process guidance. No stopping condition beyond a Done-When check.
`hardened-skills`	Seven-step structured prompt. Explicit stopping condition. Instruction to read existing tests first. Guidance on slice selection, version-aware patterns, and what *not* to test.

Table 1: Two variants tested. Both have access to the same skills. Both start from the same 6 seed files with older patterns.

What we measured

Three tiers of automated judges:

- **T0 (BuildSuccess)**: `./mvnw clean test jacoco:report` exits 0. Fail = reject immediately.
- **T1 (CoverageImprovement)**: JaCoCo line coverage ≥85% and improvement from baseline. Normalized 0–1.
- **T2 (TestQuality)**: LLM judge scoring 6 criteria with `grep`-enforced checks. Range 0.0–1.0 per criterion.

Tool calls were classified into 9 semantic states for Markov chain analysis (EXPLORE, SHELL, READ_KB, READ_SKILL, JAR_INSPECT, WRITE, BUILD, VERIFY, FIX). Journal analysis answered 6 behavioral questions about skill consumption, read order, redundant reads, and cost distribution.

2 The Smoking Gun

Every run tells the same story. Here is run n1/hardened-skills, tool calls 5–28:

```
[ 5] Skill(spring-mvc-testing)        <- reads the skill
[ 6] Read(mvc-rest-testing-patterns.md) <- reads the reference doc
    ... reads existing tests, production code ...
[28] Write(PetControllerTests.java)    <- writes with mockMvc.perform()
```

The skill teaches `MockMvcTester`. The agent reads it. The agent then reads the existing `OwnerControllerTests.java`, which uses `mockMvc.perform()`. When it writes its own controller test, it uses `mockMvc.perform()`.

This pattern repeats in **all 6 runs**, both variants. The T2 judge catches it:

T2 Criterion	Score	Passed?
test_slice_selection	1.00	Yes
assertion_quality	0.80	Yes
error_and_edge_case_coverage	0.80	Yes
domain_specific_test_patterns	**0.30**	**No**
coverage_target_selection	0.80	Yes
version_aware_patterns	**0.30**	**No**
Average (T2)	**0.667**	—

Criteria 4 and 6 fail because of the two seed-file defects:

- **version_aware_patterns = 0.30**: Zero `MockMvcTester` occurrences, 17–18 `mockMvc.perform()` calls across 5 files. The judge's `grep` enforcement: Boot 4+ projects using `mockMvc.perform()` score 0.30 max.
- **domain_specific_test_patterns = 0.30**: Zero `flush()/clear()` calls in `@DataJpaTest`. The judge's `grep`: `save()` then `findById()` without `flush()+clear()` = Hibernate L1 cache false positive.

T2 = 0.667 across all 6 runs. Both variants. Every session. The quality ceiling is set by the seed files, not by the prompt or the skills. The 4 criteria the agent passes (slice selection, assertion quality, error coverage, target selection) reflect the model's strong baseline on a known domain. The 2 it fails reflect the exemplar's pull.

3 Two-Axis Results

Variant	N	Mean cost	±	Mean turns	T2 quality	Final cov.
`simple`	3	$3.60	0.54	47.0	0.667	95.2%
`hardened-skills`	3	$3.46	0.72	46.7	0.667	92.9%

Table 2: Summary results. Cost, turns, and coverage vary across runs; T2 quality is invariant.

Run	Variant	Cost	Turns	Duration	Baseline	Final	T1	T2
n1	simple	$4.07	42	20.3 min	64.9%	95.3%	0.608	0.667
n1	hardened-skills	$3.99	52	16.9 min	64.9%	94.6%	0.595	0.667
n2	simple	$3.72	52	18.0 min	64.9%	94.6%	0.595	0.667
n2	hardened-skills	$2.62	37	12.5 min	64.9%	89.2%	0.486	0.667
n3	simple	$3.00	47	13.0 min	64.9%	95.6%	0.615	0.667
n3	hardened-skills	$3.77	51	16.4 min	64.9%	94.9%	0.601	0.667

Table 3: Per-run breakdown. All 6 runs pass T0 (build success). T2 = 0.667 is invariant. T1 varies because coverage ranges from 89.2% to 95.6%.

The n2/hardened-skills outlier: 37 turns, $2.62, 89.2% coverage. This is the cheapest run in the dataset — but also the lowest coverage. The agent wrote fewer tests and stopped earlier. It still passed T0 (build success) and cleared the 85% minimum, but produced less thorough coverage than the other 5 runs.

4 Four Key Findings

Finding 1: The exemplar controls quality, not the skill.

All 6 runs: T2 = 0.667. Both variants. The agent reads the skill (spring-mvc-testing teaches `MockMvcTester`), reads the reference document, then writes `mockMvc.perform()` because that's what the seed files demonstrate.

The skill fires in every run. It is consumed (the agent reads it). It is ignored in practice. The existing code is the stronger signal.

What the agent sees beats what the agent is told.

Finding 2: The behavioral fingerprint differs even when quality converges.

Both variants produce T2 = 0.667. But they get there differently.

Metric	simple	hardened-skills
Orientation phase	72% of calls	65% of calls
First file read	`PetClinicApplication.java`	`pom.xml`
Read order	production code first	existing tests first
Redundant reads per run	16.7	4.3
Avg tool calls	84	61
Expected steps (Markov)	260	164

`simple` reads every production file, then re-reads them when it needs to write tests. `hardened-skills` reads existing tests first (as instructed), infers what's needed, and avoids re-reading. The 37% reduction in expected steps is real efficiency — it just doesn't manifest as quality because the quality ceiling is set by the exemplar.

Finding 3: Redundant reads are the concrete inefficiency.

`simple`: 16.7 redundant file reads per run (files read $\geq$2 times). `hardened-skills`: 4.3 redundant reads per run. A 4$\times$ difference.

The `simple` agent reads production source files, writes tests, then re-reads the same files to write more tests. It does not retain file content across turns. The `hardened-skills` agent reads production files once because the prompt directs it to survey coverage gaps first, then write.

Each redundant read costs tokens (cache read tokens, but still billed) and adds a tool-call round trip.

Finding 4: PetClinic coverage converges regardless.

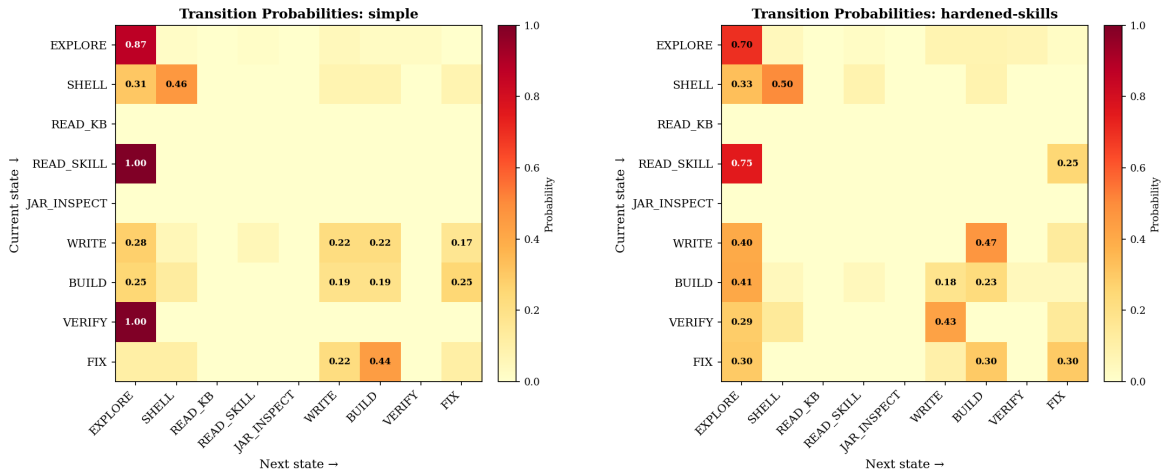
All 6 runs reach 89–96% line coverage from a 64.9% baseline. This confirms the v2 finding: Spring PetClinic is the model floor — the model knows this domain and produces adequate coverage with any prompt.

The v3 contribution is showing that coverage convergence coexists with quality divergence that the exemplar controls. The model gets the lines covered but uses the older API because the seed files demonstrate that pattern.

5 The Behavioral Signature

Two different paths to the same destination

The Markov chain analysis reveals fundamentally different navigation patterns despite identical outcomes.



simple: EXPLORE self-loop at 87%. The agent reads, re-reads, reads again. Orientation-heavy.

hardened-skills: EXPLORE self-loop at 70%. More transitions to WRITE and BUILD.

Figure 1: Transition probability heatmaps. The EXPLORE self-loop dominance in `simple` means the agent is stuck reading files rather than writing tests. `hardened-skills` exits the reading phase sooner.

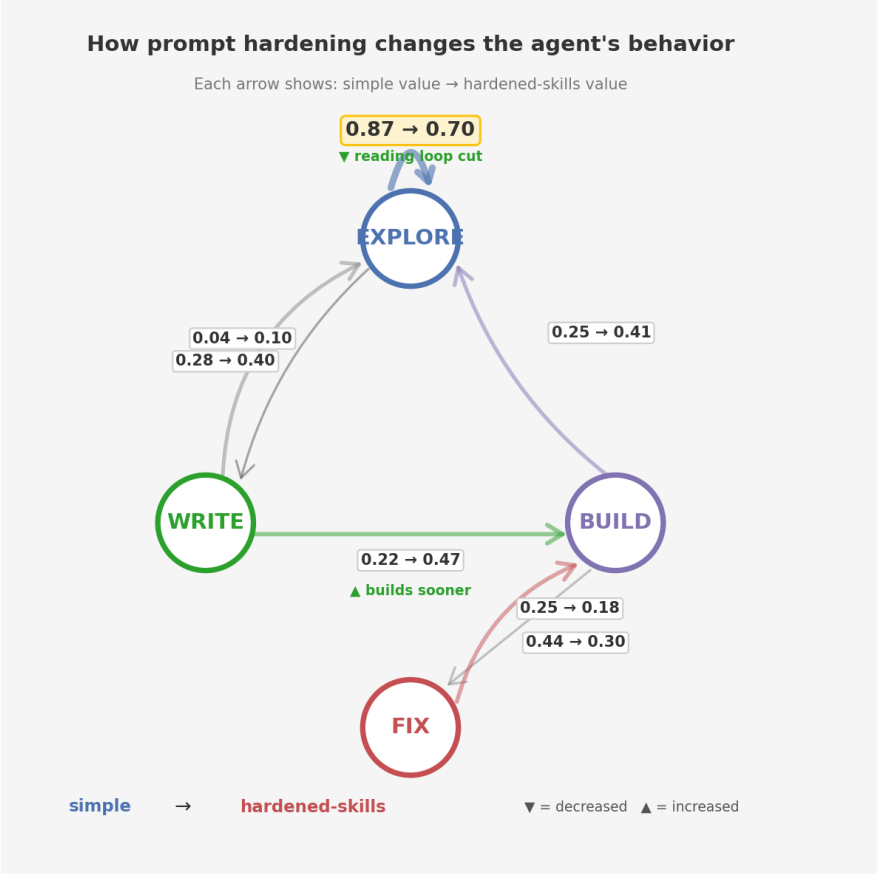


Figure 2: Combined state diagram showing both variants on a single graph. Each arrow shows: `simple` value → `hardened-skills` value. Two changes stand out: the EXPLORE self-loop drops from 0.87 to 0.70 (the agent spends less time re-reading files), and the WRITE→BUILD transition more than doubles (0.22 → 0.47) — the agent writes a test and immediately builds it instead of returning to read more files.

The Markov numbers

Variant	P(success)	Exp. steps	FIX_LOOP %	SEARCH %	JAR_INSPECT %
`simple`	1.000	260.3	11.0%	6.2%	0.0%
`hardened-skills`	1.000	164.0	20.1%	7.3%	0.0%

Both variants have P(success) = 1.0 and JAR_INSPECT = 0%. The zero JAR inspection is expected: PetClinic's dependencies are standard Spring Boot starters, and Boot 4 import paths are in the model's training data.

The 37% reduction in expected steps (260 → 164) comes from the `hardened-skills` prompt breaking the EXPLORE self-loop: the agent reads fewer files, re-reads less, and transitions to WRITE sooner.

Loop amplification: The EXPLORE state has 189× amplification in `simple` vs. 93× in `hardened-skills`. This means an agent starting in EXPLORE is expected to visit

that state 189 times before exiting in the simple variant — a direct measure of the "reading treadmill" the unstructured prompt creates.

FIX_LOOP: higher in hardened-skills

Counterintuitively, `hardened-skills` has a higher FIX_LOOP percentage (20.1% vs. 11.0%). This is because the hardened prompt includes explicit instructions to verify compilation after each batch of writes (*"Run `./mvnw compile` — do NOT proceed if compilation fails. Fix it first."*). The agent follows this instruction: it builds more often, catches errors earlier, and fixes them. The `simple` agent writes more files before attempting a build, so its FIX/BUILD ratio is lower but its failures are larger.

Both variants average 1.0 fix cycles per run — PetClinic is easy enough that neither variant spirals.

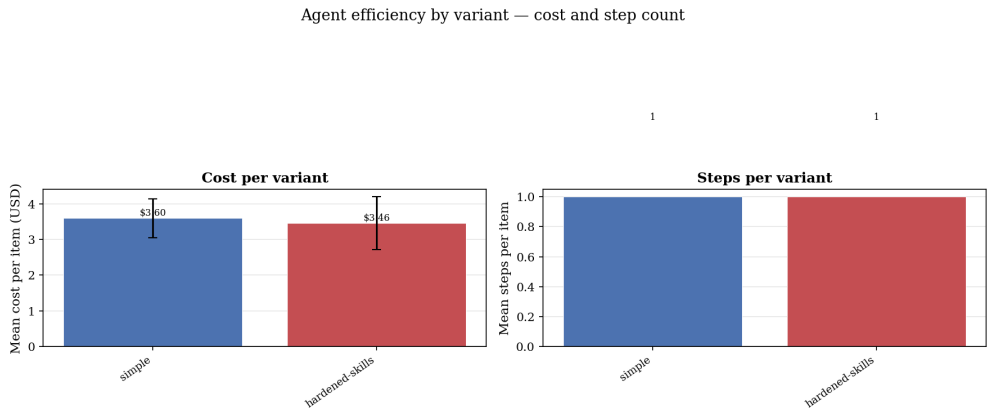


Figure 3: Cost (left) and step count (right) by variant. The difference is modest in absolute terms because PetClinic is a simple task — the behavioral divergence is larger than the cost divergence.

6 The Exemplar Effect: v2 vs. v3

The v2 experiment started from **zero tests**. No seed files. Nothing for the agent to mimic.

The v3 experiment starts from **6 seed test files** using older patterns. Skills are identical. The model is identical.

	v2 (zero tests)	v3 (seed files)	What changed
Coverage (simple)	92–94%	89–96%	Converges either way
T2/T3 quality	0.783–0.878	0.667	Exemplar patterns cap quality
version_aware	0.70–1.00	**0.30**	Seed files use `perform()`
domain_specific	0.70–0.90	**0.30**	Seeds lack flush/clear
JAR_INSPECT	1.7–17.7%	**0.0%**	Boot 4 imports are known

Table 4: v2 vs. v3 comparison. The seed files pull down the two criteria that depend on adopting improved APIs. Quality on the other 4 criteria is comparable.

The paradox: In v2, the agent sometimes discovered the correct patterns on its own (T3 scores up to 0.878). In v3, with explicit skill guidance *and* a structured prompt, the agent scores 0.667 — because the seed files are a stronger signal than the skills. Providing knowledge does not help when the codebase contradicts it.

This is the central finding of v3. The v2 experiment showed that knowledge injection (skills, KB) improves efficiency. v3 shows the boundary condition: **knowledge injection fails when existing code provides a contradictory exemplar.**

7 What This Experiment Cannot Show

Same petclinic ceiling as v2.

PetClinic is still the model floor. Coverage converges to 89–96% regardless of variant. The coverage axis is not a discriminator.

The T2 quality axis *is* a discriminator — but only because the seed files create a measurable defect. On a project with correct seed files, both variants would score identically on T2 as well.

What v3 does prove:

- The exemplar effect is real and measurable: 6/6 runs, consistent across variants and sessions.
- Behavioral fingerprints differ even when outcomes converge — observable via Markov analysis and journal inspection.
- Redundant reads are a concrete, quantifiable efficiency tax (16.7 vs. 4.3 per run).
- The model reads skills, reads references, and still follows the code. This is not a routing failure — it is a precedence failure.

What v3 does not prove:

- Whether fixing the seed files would raise T2 above 0.667. (The v2 data suggests yes: without seed files, T3 reached 0.878.)
- Whether the exemplar effect holds on codebases the model does not know. (On novel domains, the model may rely more on skills because it has less prior knowledge to fall back on.)
- Whether stronger prompt intervention ("ignore existing patterns, use MockMvcTester") would override the exemplar signal. This is a v4 variant hypothesis.

8 Implications for Practice

Three takeaways for anyone running coding agents:

1. **Fix the exemplar before optimizing the prompt.** The agent will mimic what it sees. If the existing tests use the older API, the agent will use the older API. No amount of skill guidance overrides the codebase. Update the seed files first.
2. **Behavioral differences are invisible in pass/fail metrics.** Both variants pass

every judge. Both produce ∼90–95% coverage. The 4× difference in redundant reads and the 37% difference in expected steps are only visible in the tool-call trace. If you only measure outcomes, you miss the efficiency story.

3. **Knowledge injection is an efficiency tool, not a quality tool, on known domains.** Skills reduce EXPLORE amplification from 189× to 93×. They do not improve quality when the model already knows the domain. The quality lever is the exemplar.

9 What Comes Next

Two follow-up experiments are planned:

v4: Fix the exemplar

Same task, same codebase. Replace the 6 seed test files with versions that use `MockMvcTester` and include `flush()/clear()`. Re-run both variants.

Prediction: T2 rises to ≥0.85 (matching v2's hardened quality) because the exemplar and the skill now agree. The skill becomes reinforcement rather than a contradicted suggestion.

This is the cheapest possible experiment: 6 files changed, 6 runs, same infrastructure.

Fineract: novel domain

The v2 report's planned follow-up. Apache Fineract (banking, Gradle, Spring Boot 3.2.2). Novel domain the model does not know well.

On a novel domain, the model cannot fall back on training data. Skills should have a *quality* effect, not just an efficiency effect. The seed-file exemplar effect may be weaker because the model has less certainty about what patterns to follow.

10 The Short Summary

What the v3 experiment shows:

1. **The agent reads the skill and ignores it.** 6/6 runs: skill consumed, reference read, old pattern written. The existing code is a stronger signal than structured knowledge.

2. **T2 = 0.667 is a hard ceiling set by the seed files.** 4 criteria pass (slice, assertion, error, target). 2 fail (domain patterns, version awareness). Both failures trace directly to defects in the seed test files.

3. **The behavioral fingerprint diverges even when quality converges.** `simple`: 260 expected steps, 16.7 redundant reads, production-first. `hardened-skills`: 164 expected steps, 4.3 redundant reads, test-first. A 37% efficiency difference invisible in pass/fail metrics.

4. **Fix the code, not the prompt.** Prompt hardening and skills improve how the agent navigates. They do not override what the agent imitates. The highest-leverage intervention is updating the exemplar.

The v3 punchline: Knowledge injection works — but the codebase is the agent's primary teacher. If the codebase demonstrates older patterns, no skill can override them.